数字逻辑基础与Verilog设计
---组合篇

第4章 组合电路模块

本章内容

- 多路选择器（4.1）
  - 用Verilog设计多路选择器（4.6）
- 译码器、多路分配器（4.2）
- 编码器、代码转换器与算术比较电路（4.3-4.5）
- 算术运算电路（第3章，…）

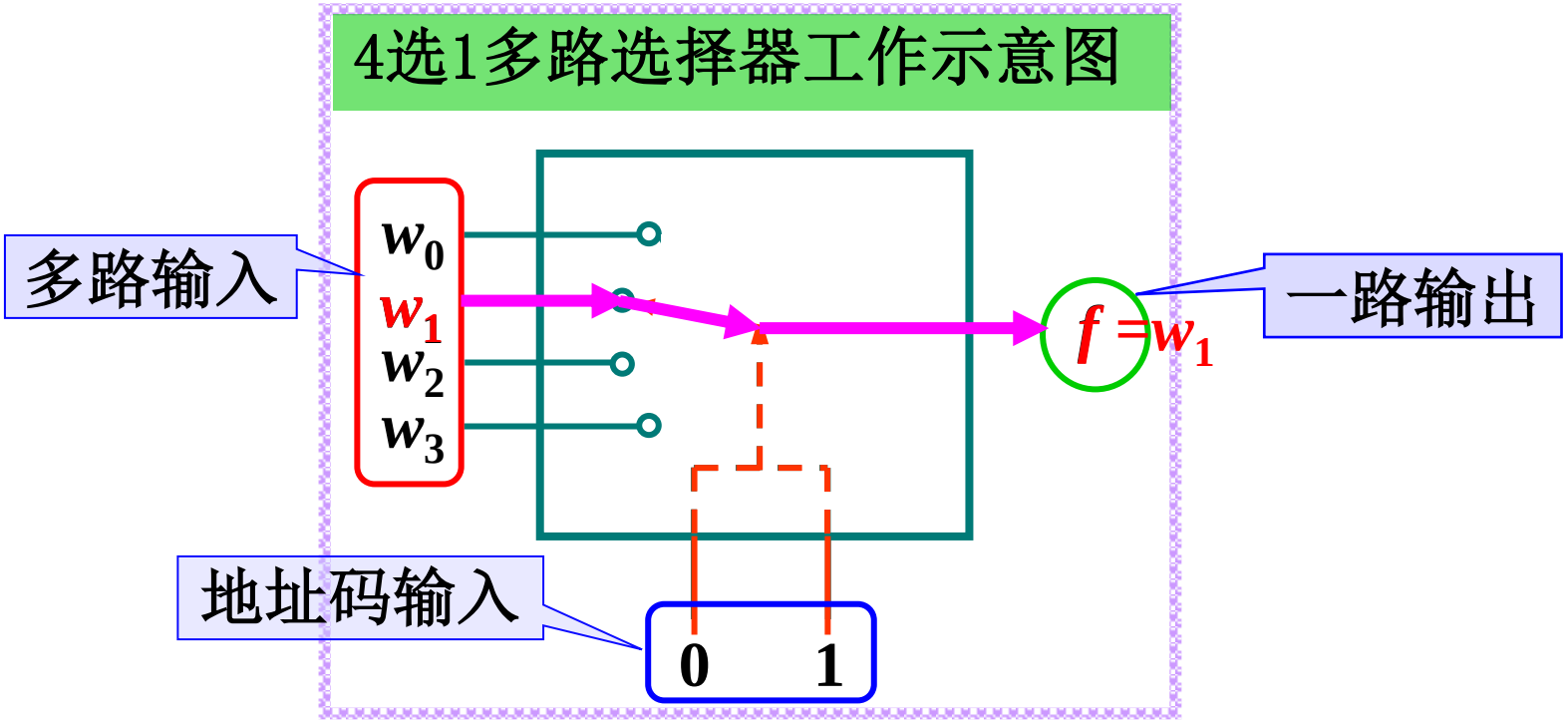
1.多路选择器

- 组合逻辑电路
  - 任何时刻产生的输出值仅取决于当前时刻的输入值，而与原来的状态无关。
  - 特点：
    - 电路的基本组成单元是逻辑门；
    - 在电路结构上具有单向性，无记忆功能、无反馈回路。
    - 由于门电路有延时，故组合逻辑电路也有延迟时间。
- 时序电路
  - 输出信号值不仅取决于当前的输入信号值，还取决于电路的历史状态。基本单元是触发器，具有反馈回路。

1.多路选择器

工作原理：

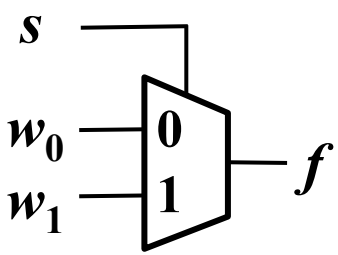
在数字信号的传输过程中，根据地址码的要求，从一组数据中选出一路输出的电路，也叫多路开关（MUX）。



地址码个数为n的数据选择器，其输入信号个数最多为2^n个。

1.多路选择器

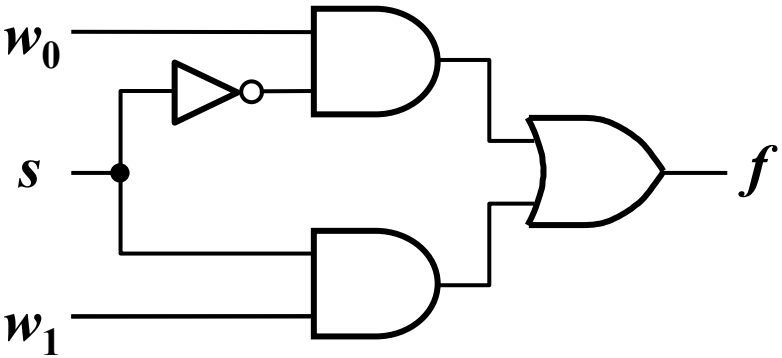
2-to-1 multiplexer：



(a) 图形符号

s	f
0	w_0
1	w_1

(b) 真值表

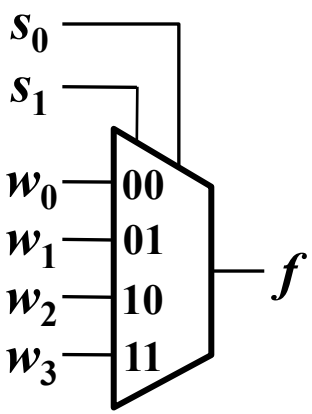


(c) 电路符号图

2-to-1 MUX表达式： $f = s'w_0 + sw_1$

1.多路选择器

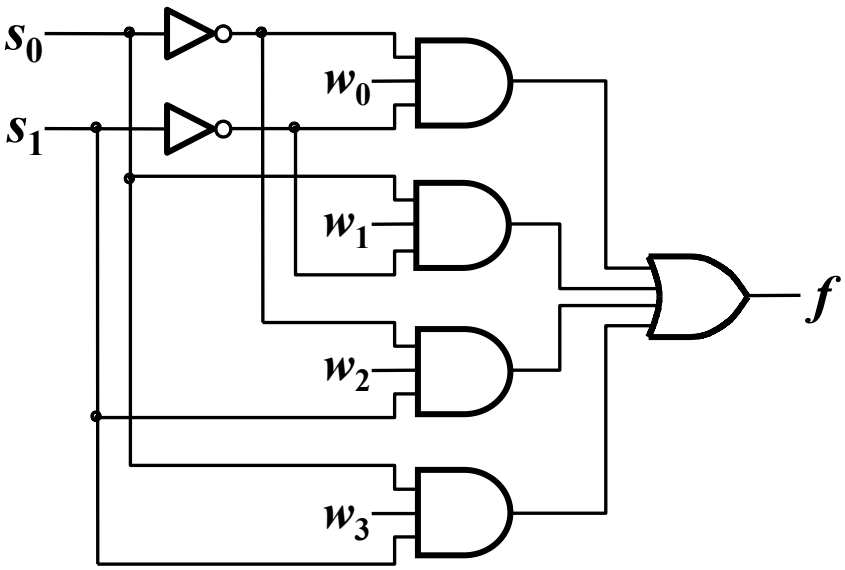
4-to-1 multiplexer：



(a) 图形符号

s_1	s_0	f
0	0	w_0
0	1	w_1
1	0	w_2
1	1	w_3

(b) 真值表



(c) 电路符号图

4-to-1 MUX表达式： $f = s'_1 s'_0 w_0 + s'_1 s_0 w_1 + s_1 s'_0 w_2 + s_1 s_0 w_3$

1.多路选择器

多路选择器的应用：

- 利用小型MUX构建大型的MUX；
- 纵横开关；
- 函数发生器；
- （桶形）移位器；
- …

1.多路选择器

利用小型MUX构建大型的MUX：

例1：采用2-1MUX构建4-1MUX

$$f = s'_1 s'_0 w_0 + s'_1 s_0 w_1 + s_1 s'_0 w_2 + s_1 s_0 w_3$$

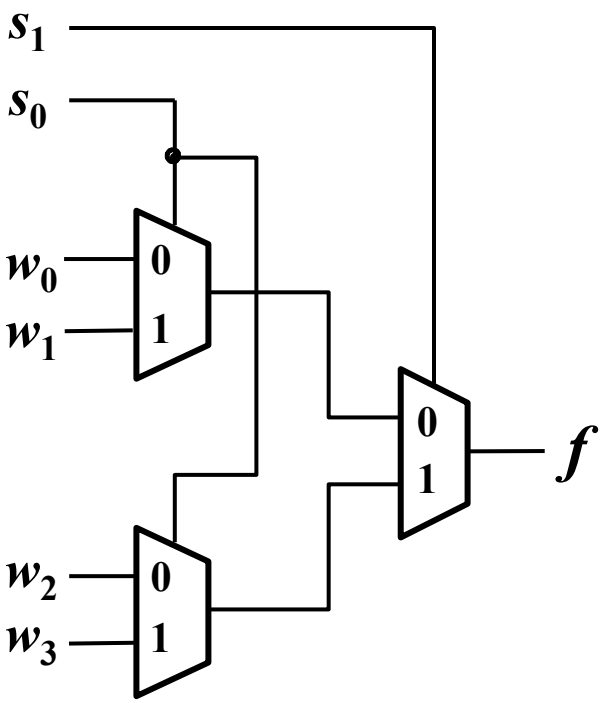
$$= s'_1 (s'_0 w_0 + s_0 w_1) + s_1 (s'_0 w_2 + s_0 w_3)$$

$$f = s'_1 g + s_1 h$$

2-to-1:

$$g = s'_0 w_0 + s_0 w_1$$

$$h = s'_0 w_2 + s_0 w_3$$



1.多路选择器

利用小型MUX构建大型的MUX：

例2：用2-1和 4-1的MUX构建8-1MUX

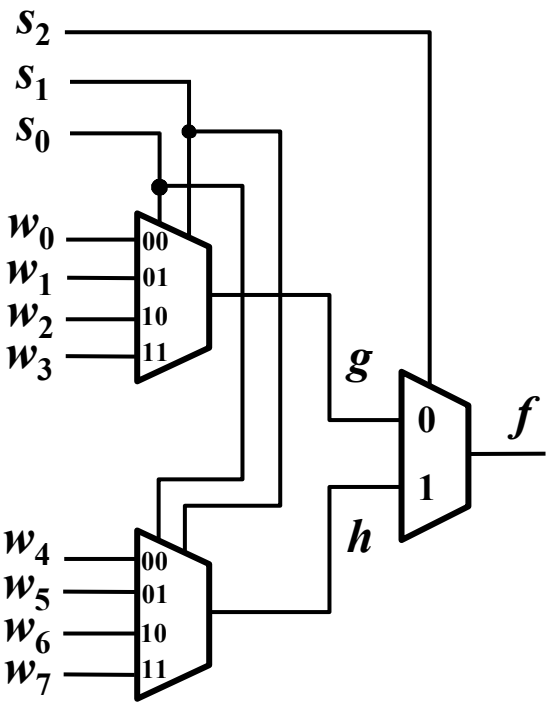
$$f = s'_2 s'_1 s'_0 w_0 + s'_2 s'_1 s_0 w_1 + s'_2 s_1 s'_0 w_2 + s'_2 s_1 s_0 w_3$$

$$+ s_2 s'_1 s'_0 w_4 + s_2 s'_1 s_0 w_5 + s_2 s_1 s'_0 w_6 + s_2 s_1 s_0 w_7$$

$$= s'_2 (s'_1 s'_0 w_0 + s'_1 s_0 w_1 + s_1 s'_0 w_2 + s_1 s_0 w_3)$$

$$+ s_2 (s'_1 s'_0 w_4 + s'_1 s_0 w_5 + s_1 s'_0 w_6 + s_1 s_0 w_7)$$

$$f = s'_2 g + s_2 h$$



同理，可以用5个4选1的MUX构建一个16-1的MUX，如P101图4.4。

1.多路选择器

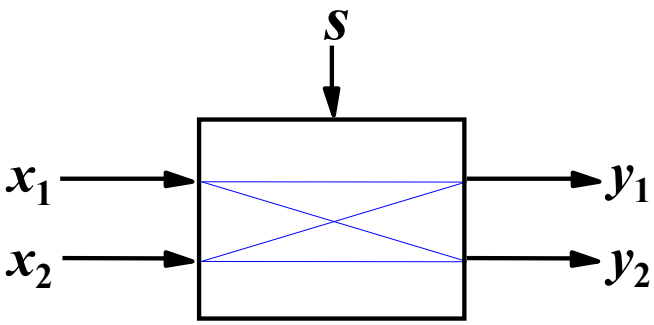
以MUX为元件的逻辑综合：

例3：用MUX实现2×2纵横开关

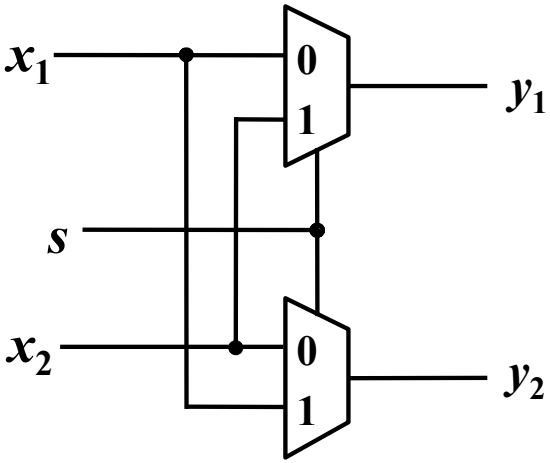
在输入信号 s 的控制下，将n个输入中的任意一个信号传输到k个输出端的任意一个输出。称为$n\times k$ 纵横开关。

$$y_1 = s'x_1 + sx_2$$

$$y_2 = s'x_2 + sx_1$$



(a) 2 x 2 纵横开关



(b) MUX实现纵横开关

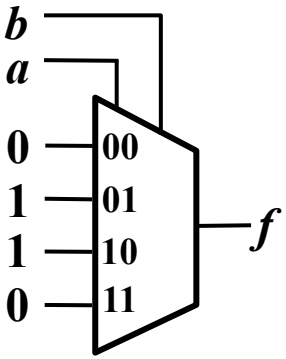
1.多路选择器

以MUX为元件的逻辑综合：

例4：用MUX实现“异或”运算

二输入的异或：$f = a'b+ab'$

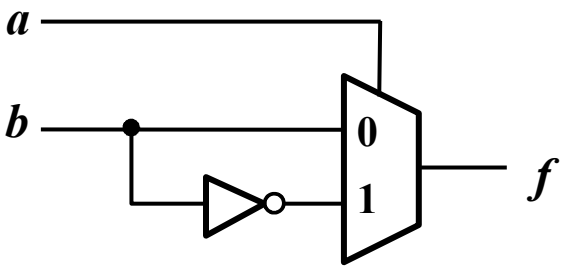
$= a'b'\cdot 0+a'b\cdot 1 +ab'\cdot 1 +ab\cdot 0$



(a) 4-to-1 mux实现xor

4-to-1: $f = s'_1 s'_0 w_0+s'_1 s_0 w_1+s_1 s'_0 w_2+s_1 s_0 w_3$

2-to-1: $f = s'w_0+sw_1$



(b) 2-to-1 mux实现xor

1.多路选择器

以MUX为元件的逻辑综合：

例4：用MUX实现“异或”运算

$$f = a \oplus b \oplus c$$

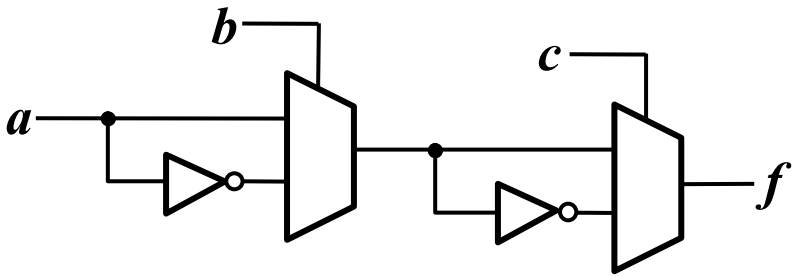


图 4.8. 3-input XOR with 2-to-1 mux

$$f = a'b'\cdot c + a'b\cdot c' + ab'\cdot c' + ab\cdot c$$

4-to-1:

$$f = s'_1 s'_0 w_0 + s'_1 s_0 w_1 + s_1 s'_0 w_2 + s_1 s_0 w_3$$

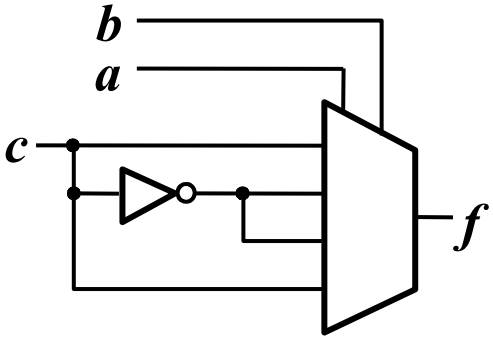


图 4.9 3-input XOR with 4-to-1 mux

1.多路选择器

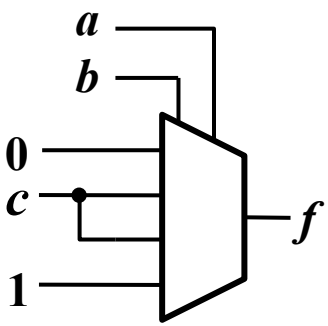
以MUX为元件的逻辑综合：

例5：用MUX实现“择多”函数

$f = a'bc+ab'c+abc'+abc$

$= a'bc+ab'c+ab$

$f = a'b'\cdot 0 + a'b\cdot c+ab'\cdot c+ab\cdot 1$

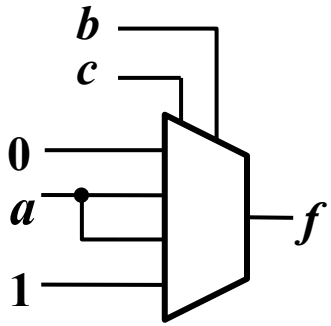


4-to-1: $f = s'_1 s'_0 w_0+s'_1 s_0 w_1+s_1 s'_0 w_2+s_1 s_0 w_3$

若以b，c为地址选择端：

$f = a'bc+ab'c+abc'+abc = b'ca+bc'a+bc$

$= b'c'\cdot 0+ b'c\cdot a+bc'\cdot a_1+bc\cdot 1$



1.多路选择器

以MUX为元件的逻辑综合：

例5：用MUX实现“择多”函数

能否用2-to-1的选择器实现？

$$f = a'bc+ab'c+abc'+abc$$

$$= a'bc+ab'c+ab$$

$$= a'bc+a(b'c+b)$$

$$= a'\cdot bc+a\cdot(c+b)$$

2-to-1: $f = s'w_0+sw_1$

a	b	c	f
0	0	0	0
0	0	1	0
0	1	0	0
0	1	1	1
1	0	0	0
1	0	1	1
1	1	0	1
1	1	1	1

a	f
0	bc
1	$b+c$

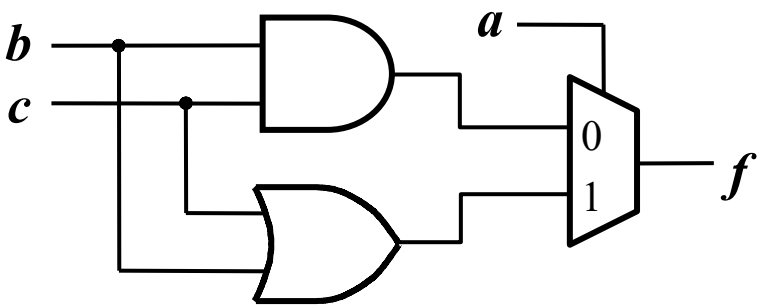


图 4.10 2-to-1 mux 实现择多函数

若以b或c为控制端，可得到不同的综合结果。

1.多路选择器

以香农展开的多路选择器综合：

定理：任何一个布尔函数 $f(w_1, w_2\cdots,w_n)$可以表示为：

$$f(w_1, w_2\cdots,w_n) = w'_1\cdot f(0, w_2\cdots,w_n) + w_1\cdot f(1, w_2\cdots,w_n)$$

上式对于n变量中的任意一个变量都有效。称：$f(0, w_2\cdots,w_n)$为w'_1的代数余子式，$f(1, w_2\cdots,w_n)$ 为w_1的代数余子式.

理解：$f(w_1, w_2\cdots,w_n) = w'_1\cdot f(w_1, w_2\cdots,w_n) + w_1\cdot f(w_1, w_2\cdots,w_n)$

$$= w'_1\cdot f(0, w_2\cdots,w_n) + w_1\cdot f(1, w_2\cdots,w_n)$$

1.多路选择器

以香农展开的多路选择器综合：

2-1MUX+门电路进行综合

例6：对函数 $f=a'c+bc'$ 分别按照 a、b、c进行分解：

$$f_1 = a'(1\cdot c+bc')+a(0\cdot c+bc')= a'\cdot(c+b) + a\cdot bc'$$

$$f_1 = a'(b'\cdot c+b\cdot 1)+a\cdot(b'\cdot 0 + b\cdot c')$$

2-1MUX进行综合

$$= a'b'\cdot c + a'b\cdot 1 + ab'\cdot 0 + ab\cdot c'$$

4-1MUX进行综合

$$f_2 = b'(a'c+0\cdot c')+b(a'c+1\cdot c')= b'\cdot a'c + b\cdot(a'+c')$$

$$f_3 = c'(a'\cdot 0+b\cdot 1)+c(a'\cdot 1 + b\cdot 0)= c'b + c\cdot a'$$

对不同变量进行展开，电路的复杂度（成本）不同.

1.多路选择器

多路选择器实现移位器：

例7：设计一个移位电路，输入为4位矢量$W=w_3w_2w_1w_0$，输出为$Y=y_3y_2y_1y_0$及信号k。当控制信号$shift=1$，将矢量W右移一位：$y_3y_2y_1y_0\,k=0w_3w_2w_1w_0$；当$shift=0$，输出$y_3y_2y_1y_0\,k=w_3w_2w_1w_00$；

分析：$y_3=shift\cdot 0+shift'\cdot w_3$，$y_2=shift\cdot w_3+shift'\cdot w_2$，…

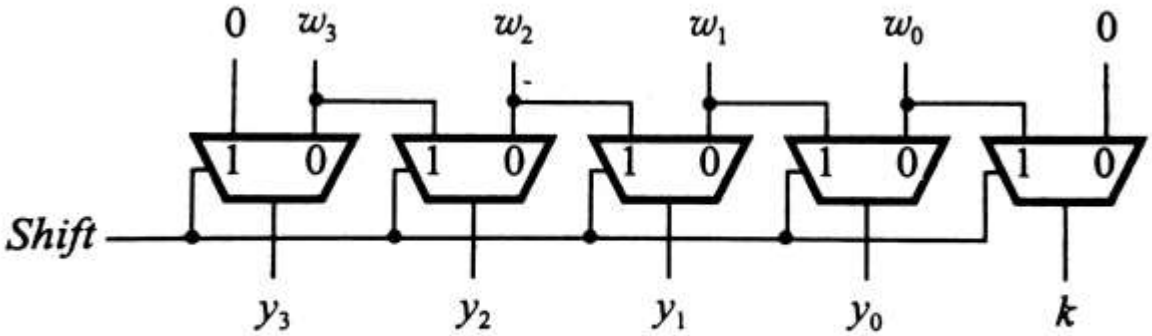


图 4.50　一个移位器电路

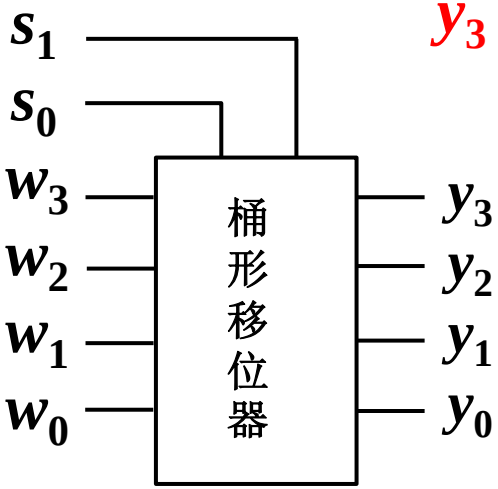
1.多路选择器

多路选择器实现移位器：

例8：桶形移位器：实现按照指定数目的位数及位置对输入矢量进行循环移位。设计一个四位桶形移位器。

分析：设定两个控制信号s_1，s_0决定循环移动0，1，2，3位.

$$y_3 = s'_1 s'_0 \cdot w_3 + s'_1 s_0 \cdot w_0 + s_1 s'_0 \cdot w_1 + s_1 s_0 \cdot w_2$$



(a) 示意图

s_1	s_0	y_3	y_2	y_1	y_0
0	0	w_3	w_2	w_1	w_0
0	1	w_0	w_3	w_2	w_1
1	0	w_1	w_0	w_3	w_2
1	1	w_2	w_1	w_0	w_3

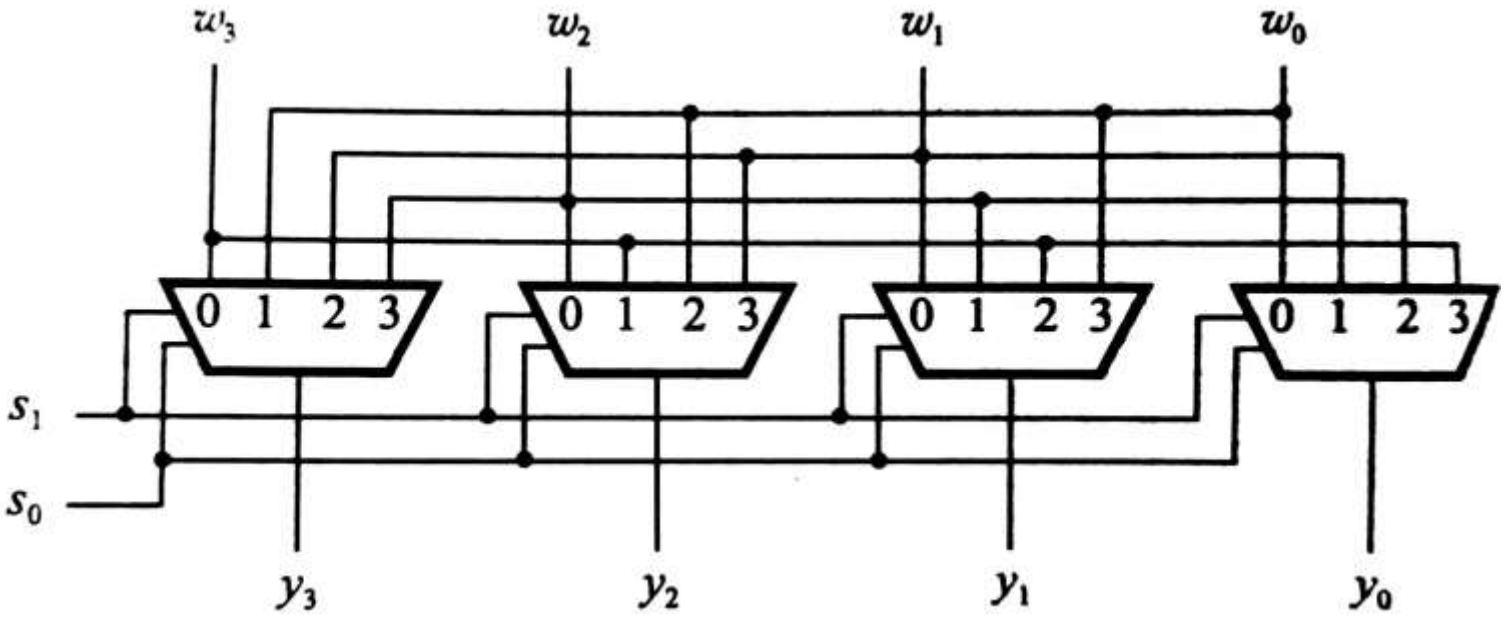
(b) 真值表

1.多路选择器

多路选择器实现移位器：

$$y_3 = s'_1 s'_0 \cdot w_3 + s'_1 s_0 \cdot w_0 + s_1 s'_0 \cdot w_1 + s_1 s_0 \cdot w_2$$

$$y_2 = s'_1 s'_0 \cdot w_2 + s'_1 s_0 \cdot w_3 + s_1 s'_0 \cdot w_0 + s_1 s_0 \cdot w_1$$



(c) 由4-1数据选择器实现的桶形移位器

2.用Verilog设计多路选择器

主要从行为级描述组合逻辑电路：

- 主要知识要点：
  - 数据类型;
  - 运算符：条件运算符
  - 条件语句：If-else语句;
  - 多分支选择语句： Case语句

2.用Verilog设计多路选择器

(1)数据类型

- 为了表示数字电路硬件中的数据存储和传送元素，共定义了19种数据类型，只介绍其中最基本的四种类型：reg型，wire型，integer型和parameter型。

2.用Verilog设计多路选择器

(1) 数据类型

- 常数（包括数字、不定值x、高阻z）
  - 完整的数字格式：<位宽>'<进制符号><数字>

    其中：位宽表示数字对应的二进制的位宽；进制符号包括b、d、h、o。（分别为二、十、十六、八进制）

    例：8'b10110001//位宽为8位的二进制数，等价8’hb1；
  - 不定值x和高阻z（或者？）：

    例：4'b100x //位宽为4的二进制数，最低位为不定值

    8'h2x //位宽为8的十六进制数，低四位为不定值

    12'dz //位宽为12的十进制数，其值为高阻态。

2.用Verilog设计多路选择器

(1) 数据类型

- 参数型（符号常量）
  - 语法：**parameter 参数名=常数表达式;**
  - 采用标识符代表一个常数，可以提高代码的可读性；一般在程序前部定义。
  - 表达式中只能包含常数或定义过的参数；
  - 参数型经常用来定义延迟时间和变量宽度。

    例： **parameter e=8, f=e-1；** //用常量表达式赋值

    parameter fbus = 8'b11010001;

2.用Verilog设计多路选择器

(1) 数据类型

- 变量
  - 变量是在程序运行时其值可以改变的量。在Verilog HDL中，变量分为：
    - 网络型（nets type）
    - 寄存器型（register type）
    - 存储器型（memory type）

2.用Verilog设计多路选择器

(1) 数据类型

– 网络型变量（nets type）

- nets型变量是输出值始终根据输入变化而更新的变量，它一般用来定义硬件电路中的各种物理连线。必须受到门或者assign驱动，否则为高阻态z。
- Verilog HDL提供的最常用的nets型变量-wire。

2.用Verilog设计多路选择器

(1) 数据类型

– 网络型变量（nets type）— wire型：

- wire型数据表示用以assign关键字指定的组合逻辑信号。
- 输入、输出信号默认为wire型。
- 可以做任何式子的输入，也可以做assign语句或实例元件的输出。
- 语法：wire [n-1:0] data1，data2，…，datai；

  wire [n:1] data1，data2，…，datai；

  // i条位宽为n的数据总线
- 例：wire a;　　//定义1个1位的wire型数据

  wire [7:0] c, d;　//定义2个8位的wire型数据

2.用Verilog设计多路选择器

(1) 数据类型

– 寄存器型变量（ **register type**）：

- **integer、real和time等3种寄存器型变量都是纯数学的抽象描述，不对应任何具体的硬件电路。**

类型	功能说明
reg	常用的寄存器型变量
integer	**32**位带符号整数型变量
real	**64**位带符号实数型变量
time	无符号时间型变量

2.用Verilog设计多路选择器

(1) 数据类型

– 寄存器型变量（ **register type**）：

- **reg**型变量是一种数值容器，不仅可容纳当前值，也可保持历史值，这一属性与触发器的记忆功能对应。
- **reg**型与**wire**型变量的区别：**reg**型数据的默认初始值为不定值**x**；因此需要被明确地赋值（相当于改变触发器的存储值），并且在被重新赋值前一直保持原值。**always**、**initial**模块中通过过程语句赋值的每一个信号必须定义为**reg**型。

2.用Verilog设计多路选择器

(1) 数据类型

– 寄存器型变量（ register type）— reg型：

- 语法：reg [n-1:0] data1, data2, …, datai;

— 默认位宽是1，称为标量，位宽超过1的变量称为向量。

— 例：reg a，b；　　//定义两个reg型变量a，b

reg[7:0] data；　//定义1个8位寄存器型变量

— 向量定义后可以采有多种赋值形式：

```
data=8'b00000000;
data[5:3]=3'b111;
data[7]=1;
```

2.用Verilog设计多路选择器

(1) 数据类型

– 存储器型变量（ memory type）：

- 通过对reg型变量建立数组来对存储器建模。
- 语法：reg [n-1:0] 存储器名[m-1:0];

例：　reg[7:0] mymemory[1023:0]；

定义了一个1024 个字存储器变量mymemory，每个字的字长为8位。可以用下面的语句来使用存储器：

```
mymemory[7] = 75;
//存储器mymemory的第7个字被赋值75
```

2.用Verilog设计多路选择器

(2) 条件运算符

- 语法：操作数 =条件_表达式?真值_表达式:假值表达式;
  - 3个运算数;可以嵌套。
  - 既可用于assign赋值语句，也可用于always过程语句中
  - 例：A 2-to-1 multiplexer

```
module mux2to1 (
    input   w0, w1, s,
    output  f
);
    assign  f = s ? w1 : w0;
endmodule
```

```
module mux2to1 (
    input   w0, w1, s,
    output  reg  f
);
    always@(s,w1, w0)
        f = s ? w1 : w0;
endmodule
```

2.用Verilog设计多路选择器

(2) 条件运算符

– 例：A 4-to-1 multiplexer

4-to-1: $f = s'_1 s'_0 w_0 + s'_1 s_0 w_1 + s_1 s'_0 w_2 + s_1 s_0 w_3$

$= s'_1(s'_0 w_0 + s_0 w_1) + s_1(s'_0 w_2 + s_0 w_3)$

```
module mux4to1 (
    input   w0, w1, w2, w3,
    input   [1:0] s,
    output  f
);
    assign  f = s[1] ? (s[0] ? w3 : w2)
                     : (s[0] ? w1 : w0);
endmodule
```

2.用Verilog设计多路选择器

(3) IF-ELSE 语句

- 语法：**if（条件_表达式） 语句1；**
  else 语句2；
  - **If-else为顺序语句，应放在always块中，语句块中的语句为多条时，需要用 begin 和 end关键字；**
  - 可以嵌套。
  - 例：**A 2-to-1 multiplexer**

```
module mux2to1 (
    input   w0, w1, s,
    output  reg  f
);
    always@(s,w1, w0)
        if (s==0)
            f = w0;
        else
            f = w1;
endmodule
```

2.用Verilog设计多路选择器

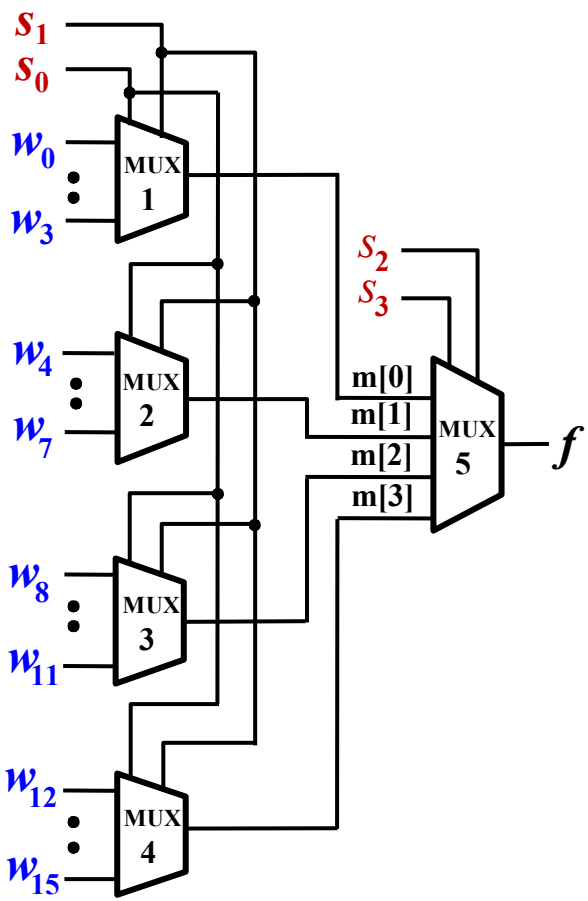
- 例：采用if-else嵌套实现 A 4-to-1 multiplexer

```
module mux4to1 (
    input w0, w1, w2, w3,
    input [1:0] s,
    output reg  f
);
    always @(*)
        if (s == 2'b00)
            f = w0;
        else if (s == 2'b01)
            f = w1;
        else if (s == 2'b10)
            f = w2;
        else if (s == 2'b11)
            f = w3;
endmodule
```

```
module mux4to1 (
    input [3:0] w,
    input [1:0] s,
    output reg  f
);
    always @(w, s)
        if (s == 0)
            f = w[0];
        else if (s == 1)
            f = w[1];
        else if (s == 2)
            f = w[2];
        else if (s == 3)
            f = w[3];
endmodule
```

2.用Verilog设计多路选择器

- 例：采用层次化设计，实现 A 16-to-1 multiplexer




```
module mux16to1 (
        input [0:15] w,
        input [3:0] s,
        output f
);
        wire [0:3] m;
        mux4to1 Mux1 (w[0:3], s[1:0], m[0]);
        mux4to1 Mux2 (w[4:7], s[1:0], m[1]);
        mux4to1 Mux3 (w[8:11], s[1:0], m[2]);
        mux4to1 Mux4 (w[12:15], s[1:0], m[3]);
        mux4to1 Mux5 (m[0:3], s[3:2], f );
endmodule
```

2.用Verilog设计多路选择器

(4) CASE 语句

- 语法：**case (表达式)**

  选择值1 : 语句1;

  …

  选择值n : 语句n;

  [default : 语句n+1;]

  endcase

case语句有两种变体语句:casez 和casex

- 不建议使用casex；
- 自学三者的异同

  - case也为顺序语句，应放在always块中；
  - 当指定的n个选择不能包含表达式所有的可能值，就必须包含default语句，否则编译器会综合出未定义情况。

2.用Verilog设计多路选择器

- 例：采用case语句实现 A 4-to-1 multiplexer

```
module mux4to1(
    input   a, b, c, d,
    input   s1, s2,
    output  reg  f
);
    always @(s1,s2,a,b,c,d)
      case ({s1,s2})
        2'b00: f=a;
        2'b01: f=b;
        2'b10: f=c;
        2'b11: f=d;
      endcase
endmodule
```

```
module mux4to1 (
      input [0:3]  w,
      input [1:0]  s,
      output reg  f
);
      always @(w, s)
            case (s)
                  0: f = w[0];
                  1: f = w[1];
                  2: f = w[2];
                  3: f = w[3];
            endcase
endmodule
```

2.用Verilog设计多路选择器

总结

- 实现选择器：case语句、if-else语句、条件运算符。
- case与if语句需要用到always语句，组合逻辑却要将输出变量声明称reg型。因此，最好还是用条件运算符实现多路选择器。
- 例如实现5选1选择器：

```
module Mux5_8b（
    input  [7:0] in0,in1,in2,in3,in4,
    input  [2:0] sel,
    output [7:0] out
);
assign out =（sel==3’d0）? in0:
             （sel==3’d1）? in1:
             （sel==3’d2）? in2:
             （sel==3’d3）? in3:
             （sel==3’d4）? in4:
                            8’b0;
endmodule
```

2.用Verilog设计多路选择器

- 上述代码存在优先级关系
- 还可以采用如下代码实现多路选择器

  其中：

  - {{}}为复制运算符
  - “&”为按位“与”
  - “|”为按位“或”

```
module Mux5_8b（
    input  [7:0] in0,in1,in2,in3,in4,
    input  [2:0] sel,
    output [7:0] out
);
assign out =({8{sel==3'd0}}&in0)
           |({8{sel==3'd1}}&in1)
           |({8{sel==3'd2}}&in2)
           |({8{sel==3'd3}}&in3)
           |({8{sel==3'd4}}&in4);
endmodule
```

设计桶形移位器

要求：

- 实现32位桶形移位器；
- 具有算术右移、算术和逻辑左移、逻辑右移，保持四个功能。

```
module barrelshifter32_2(
                    input [31:0] a,
                    input [4:0] shift_bit,
                    input [1:0] aluc,
                    output reg [31:0] odata
);
reg [31:0] tmp;
always@ *
begin
   if(aluc==0) //算术右移
   begin
      tmp = shift_bit[0] ? {a[31],a[31:1]} : a;
      tmp = shift_bit[1] ? {{(2){tmp[31]}}, tmp[31:2]} : tmp;
      tmp = shift_bit[2] ? {{(4){tmp[31]}}, tmp[31:4]} : tmp;
      tmp = shift_bit[3] ? {{(8){tmp[31]}}, tmp[31:8]} : tmp;
      tmp = shift_bit[4] ? {{(16){tmp[31]}}, tmp[31:16]} : tmp;
   end
   else if (aluc==1 ) //算术和逻辑左移
   begin
      tmp = shift_bit[0] ? { a[30:0],1'b0} : a;
      tmp = shift_bit[1] ? { tmp[29:0], 2'b0} : tmp;
      tmp = shift_bit[2] ? { tmp[27:0], 4'b0} : tmp;
      tmp = shift_bit[3] ? { tmp[23:0], 8'b0} : tmp;
      tmp = shift_bit[4] ? { tmp[15:0], 16'b0} : tmp;
   end
   else if (aluc==2 ) //逻辑右移
   begin
      tmp = shift_bit[0] ? { 1'b0,a[31:1] } : a;
      tmp = shift_bit[1] ? { 2'b0,tmp[31:2] } : tmp;
      tmp = shift_bit[2] ? { 4'b0,tmp[31:4]} : tmp;
      tmp = shift_bit[3] ? { 8'b0,tmp[31:8]} : tmp;
      tmp = shift_bit[4] ? { 16'b0,tmp[31:16]} : tmp;
   end
   else  //不变
      tmp = a;
   odata = tmp;
end
endmodule
```

本章作业：

- **4.1（或4.2），4.3（或4.4），4.5，4.8，4.11；**
- **4.16，4.18，4.19，4.21，4.22，4.25，**
- **4.26，4.27，**
- **4.29，4.30，4.32，4.33**